

A5 기술 실현 수준 보고서

작성 기준: 2026-05-25 14:16 KST 대상 경로: C:\Users\djfh1\Desktop\my-app 판
단 기준: 현재 로컬 코드, 기존 상태 문서, 방금 실행한 검증 명령 결과

1. 종합 판정

A5는 현재 **상용 운영 직전 단계가 아니라, 폐쇄물 서비스의 화면/데이터 구조/통합 경계가 상당 부분 구현된 베타 기술 검증 단계**입니다.

정리하면 다음 수준까지 실현되어 있습니다.

영역	현재 수준	판정
고객/태블릿/관리자 화면	주요 업무 화면 대부분 구현	데모 가능
상품 조회	Firestore products 읽기 + mock fallback 구현	제한적 실연 가능
CMS/콘텐츠 관리	Firestore/Storage 연동 코드 구현	권한/계정 준비 후 검증 필요
Firebase 기반	Web SDK, rules, seed script, repository 구현 진행	베타 연결 단계
PG 결제	브라우저/Functions 계약, mock confirm skeleton 구현	실제 결제 불가
주문/결제/재고 트랜잭션	설계와 skeleton 중심	운영 불가
배포/QA	route/env/secret gate 일부 통과	현재 build/lint 실패로 릴리즈 불가
외부 연동	Alimtalk, 배송, 외부 재고, A4는 mock/계약 대기	미실현

전체 체감 완성도는 **데모/MVP 기준 60~70%**, 실제 결제와 정산이 들어가는 **운영 기준 35~45%**로 보는 것이 적절합니다. 화면과 흐름은 꽤 넓게 열려 있지만, 돈과 권한과 외부 API가 걸리는 구간은 아직 차단되어 있습니다.

2. 구현 완료 또는 실연 가능한 부분

2.1 화면/라우트

`node scripts\check-routes.mjs` 기준 **App Router page route 69개**가 존재하고 필수 smoke route는 모두 존재합니다.

구현된 주요 화면 범위는 다음과 같습니다.

- 고객/태블릿: `/products`, `/tablet/products`, `/tablet/products/[id]`, `/tablet/cart`, `/tablet/qr`, `/q/[code]`, `/q/[code]/checkout`
- 비회원 주문: `/orders/guest`, `/orders/guest/[orderNo]`, `/orders/guest/[orderNo]/refund`
- 최고관리자: dashboard, companies, nurseries, rooms, tablets, products, orders, payments, settlements, audit logs
- 기업 관리자: products, orders, inventory, deliveries, sales, payouts, ads, brand, exhibitions
- 조리원 관리자: dashboard, rooms, tablets, pickups, qr-history, orders
- mock 검증 화면: `/mock-ui/status`, `/mock-ui/smoke`, `/mock-ui/storefront` 등

따라서 **서비스 설명용 화면 데모와 업무 흐름 시연은 가능합니다.**

2.2 상품 데이터

`lib/repositories/firebase/firebaseProductRepository.ts` 에서 Firestore `products` 컬렉션의 `status == "active"` 상품을 우선 읽고, 실패/빈 결과/미설정 시 mock fallback으로 전환합니다.

현재 실현 수준:

- `/products`, `/tablet/products`, `/tablet/products/[id]` 가 Firebase 상품 읽기 경로를 가짐
- 상품 `id/status/source/seeded_at` 표시 가능

- Firestore 연결 실패 시 화면이 깨지지 않도록 mock fallback 유지
- product seed script 존재

즉, **상품 조회는 Firebase 베타 수준까지 구현**되어 있습니다.

2.3 Firebase CMS/Storage

`lib/firebase/contentRepository.ts` 기준으로 CMS 레코드 구독/저장 및 파일 업로드 helper가 구현되어 있습니다.

구현된 컬렉션:

- `marketing_banners`
- `marketing_videos`
- `product_detail_pages`
- `home_sections`
- `tablet_home_configs`
- `media_assets`

Storage 업로드 경로도 회사/상품/스토어프론트 맥락에 맞게 분기됩니다.

다만 이 기능은 **Firebase Web config, 로그인 사용자, Custom Claims, rules 권한이 맞아야 실제 검증** 가능합니다. 코드 연결은 되어 있으나 운영 권한 체계가 아직 완성되지 않았습니다.

2.4 Firebase Functions/PG skeleton

`functions/src/index.ts`에 다음 HTTPS Functions export가 준비되어 있습니다.

- `paymentsReady`
- `paymentsConfirm`
- `paymentsWebhook`
- `paymentsCancel`

`npm.cmd --prefix functions run build` 결과 Functions TypeScript 빌드는 통과했습니다.

`paymentsConfirm`은 현재 다음을 수행하는 skeleton입니다.

- POST 요청 검증

- QR session 활성화 상태 검증 skeleton
- 클라이언트 금액과 서버 재계산 금액 비교
- 주문번호 생성
- 재고 차감 계획 생성
- 주문 snapshot write plan 생성
- audit log write plan 생성
- mock PG 승인 응답 반환

중요: 실제 PG API 호출, 실제 Firestore transaction write, 실제 재고 차감은 아직 수행하지 않습니다.

3. 현재 검증 결과

2026-05-25 14:16 KST 기준으로 직접 실행한 결과입니다.

명령	결과	의미
npm.cmd run check:env	통과	필수 NEXT_PUBLIC_FIREBASE_* 키 존재 확인, 값 출력 없음
npm.cmd run check:no-secrets	통과	.env.local은 로컬에만 있고 git index 미추적, serviceAccountKey.json 없음
node scripts\check-routes.mjs	통과	69개 page route, 필수 smoke route 모두 존재
npm.cmd --prefix functions run build	통과	Functions TypeScript 컴파일 가능
npm.cmd run lint	실패	functions/lib 생성 JS가 ESLint 대상에 포함되어 CommonJS require 오류 22건 발생, 경고 12건
npm.cmd run build	실패	firebaseOrderRepository.ts 의 "refunded" 상태가 OrderStatus 타입에 없어 type check 실패

이 때문에 현재 커밋 상태 그대로는 릴리즈/배포 가능 상태가 아닙니다.

4. 현재 막힌 핵심 기술 경계

4.1 빌드 차단

lib/repositories/firebase/firebaseOrderRepository.ts 의 asOrderStatus() 허용 목록에 "refunded" 가 있으나, types/status.ts 의 OrderStatus 에는 "refunded" 가 없습니다.

즉시 조치 방향:

- 실제 상태 모델에서 refunded 가 필요하면 OrderStatus 와 라벨/tone에 추가

- 아니면 repository의 허용 목록에서 `refunded` 를 제거하고 기존 `refund_approved_mock` 등으로 매핑

이 문제를 해결하기 전까지 `next build` 는 통과하지 않습니다.

4.2 lint 차단

`functions/lib/**` 는 Functions TypeScript 빌드 산출물입니다. 현재 루트 ESLint가 이 생성 JS를 검사하면서 `require()` import 금지 규칙에 걸립니다.

즉시 조치 방향:

- `eslint.config.mjs` 의 ignore에 `functions/lib/**` 추가
- 또는 Functions 산출물 디렉터리를 루트 lint 범위 밖으로 분리

별도로 `<img>` 사용 경고 12건도 남아 있습니다. 이는 실패 원인은 아니지만 이미지 전략 결정 후 정리하는 것이 좋습니다.

4.3 실제 결제/주문 운영 차단

PG 연동은 현재 계약과 skeleton 수준입니다.

아직 없는 것:

- PG 공식 provider 확정
- sandbox MID/client key/channel key/secret key
- webhook signature 검증 규칙
- 실제 `confirm/cancel/refund` API 호출
- PG 성공 후 Firestore transaction write
- 결제 중복/만료/금액 불일치/재고 부족 케이스의 운영 테스트

따라서 **결제는 mock 데모만 가능합니다.**

4.4 Firebase 권한/계정 차단

Custom Claims와 운영 계정이 아직 준비되지 않았습니다.

필요 claims:

- `SUPER_ADMIN`
- `COMPANY_ADMIN`

- NURSERY_ADMIN
- TABLET_DEVICE
- seed_admin
- company_id, nursery_id, room_id, tablet_id

이 권한 체계가 준비되어야 CMS write, seed 실행, 운영자 화면의 실제 write를 안전하게 열 수 있습니다.

4.5 외부 시스템 차단

다음은 아직 mock 또는 문서 대기 상태입니다.

- Alimtalk/SMS
- 배송 조회/송장 API
- 외부 재고 API
- A4 조리원 시스템 mapping/API
- 정산/지급/환불 운영 정책

5. 기술 실현 단계별 평가

단계	설명	현재 상태
1단계: 화면 prototype	주요 사용자별 화면과 mock 데이터 구성	완료에 가까움
2단계: mock 업무 흐름	QR, 장바구니, 주문조회, 관리자 콘솔 mock flow	완료에 가까움
3단계: Firebase read beta	상품 조회를 Firestore 우선으로 연결	구현됨
4단계: Firebase write beta	CMS/Storage write 코드 연결	구현됐지만 claims 검증 필요
5단계: 서버 결제 skeleton	Functions PG ready/confirm/webhook/cancel 구조	구현됨, mock only
6단계: 실제 주문 transaction	PG confirm 후 주문/결제/재고 원자 처리	미완료
7단계: 운영 배포	lint/build/release gate/Cloudflare/Firebase deploy	현재 실패/미완료
8단계: 상용 운영	PG, 환불, 정산, 알림, 배송, 외부 재고	미실현

현재 A5는 **4~5단계에 걸쳐 있는 상태**입니다. 화면과 Firebase 일부 연결은 이미 구현되었지만, 상용 운영의 핵심인 결제 확정, 주문 기록, 재고 차감, 환불/정산은 아직 진입 전입니다.

6. 다음 우선순위

1. `OrderStatus` 와 Firebase order repository 상태값 불일치를 수정해 `npm.cmd run build` 를 복구합니다.

2. `functions/lib/**`를 루트 ESLint ignore에 추가해 생성 산출물 때문에 lint가 실패하지 않게 합니다.
3. `npm.cmd run lint`, `npm.cmd run build`, `npm.cmd --prefix functions run build`, `node scripts\check-routes.mjs`를 다시 통과시킵니다.
4. seed/admin 계정에 `seed_admin` 또는 `SUPER_ADMIN` claim을 부여한 뒤 foundation seed를 실행합니다.
5. `/admin/marketing/banners`, `/company/products/preview`에서 실제 Firestore/Storage write를 1건씩 검증합니다.
6. PG provider 문서와 sandbox 키를 받은 뒤 Functions의 mock confirm을 실제 provider confirm adapter로 교체합니다.
7. PG 성공 후 Firestore transaction으로 `orders`, `order_items`, `payments`, `payment_events`, `inventory`를 원자 처리합니다.
8. 환불/취소/정산/알림/배송/A4 연동은 운영 정책과 API 계약 확정 후 별도 단계로 엽니다.

7. 결론

A5는 현재 “**볼 수 있는 폐쇄물**”과 “**일부 Firebase가 연결된 베타**” 수준까지는 도달했습니다. 특히 고객/태블릿/관리자/기업/조리원 화면 범위가 넓고, 상품 조회와 CMS 저장 경로, PG Functions skeleton까지 마련되어 있어 기술 방향은 잡혀 있습니다.

하지만 현재 시점의 냉정한 결론은 **상용 결제 운영 가능 상태는 아닙니다**. 먼저 build/lint를 복구하고, Firebase claims와 PG sandbox를 확보한 뒤, 주문/결제/재고 transaction을 Functions에서 실제로 처리하는 단계까지 가야 운영 베타라고 부를 수 있습니다.